

COS 214 Practical 5 (Campus Guard)

29 September 2026

Jay Macaskill – 25198387

Mohammad H Jafari – 25312040

Emmanuel Boateng – 23586975

https://github.com/MH-Multy/COS214_prac5

System Description

CampusGuard is an emergency-response coordination system for a university campus. It:

- Registers incidents
- Dispatches response teams (including Security, Medical and Facilities teams)
- Locks/unlocks campus areas
- Sends alerts (which can also be done through an incompatible legacy system)
- Resolves incidents
- Coordinates between teams centrally using Mediator (as opposed to direct references)

Six GoF patterns are used:

1. Command (operator actions are undoable, loggable objects)
2. Mediator (teams can coordinate through a central mediator)
3. Adapter (integrates legacy alert system, where the legacy alert system accepts an int, whilst the facing alert system accepts an AlertType Enum)
4. Facade (single entry-point for workflow)
5. Observer (broadcasts status changes to independent observers)
6. Strategy (severity-based dispatch policies)

Task 1

GoF Design Patterns & Participants Appearing in the Design

Pattern	Participants
Command	• Command (OperatorCommand) • ConcreteCommand (DispatchUnit, SecureArea, Send Alert) • Invoker (OperatorConsole) • Receiver (ResponseUnit, AccessControlSystem, AlertSender) • Client (CampusGuard)
There are three concrete commands each with a distinct receiver type. The invoker makes its execute(), undo() and name() calls polymorphically. CampusGuard, the client, makes concrete commands and passes them to the Invoker, it does not execute them directly.	
Mediator	• Mediator (IncidentCoordinator) • ConcreteMediator (IncidentResponseDesk) • Colleague (ResponseUnit) • ConcreteColleague (SecurityTeam, MedicalTeam, FacilitiesTeam)
Mediator holds non-owning pointers to concrete colleagues and routes events between them. Colleagues do not reference each other directly. The issue this pattern solves is coordination between the colleagues.	
Adapter	• Adapter (LegacyAlertAdapter) • Adaptee (LegacyAlertSystem) • Target (AlertSender) • Client (SendAlert)
The target interface is notify(AlertType) whilst the adaptee interface is sendLegacyAlert(int). The adapter translates the AlertType Enum (modern) into the legacy integer code, then forwards the call. The adapter is invisible to the client; it only knows the AlertSender interface.	
Facade	• Facade (CampusGuard) • Subsystem Classes OperatorConsole, IncidentResponseDesk, SecurityTeam, MedicalTeam, FacilitiesTeam, AccessControlSystem, LegacyAlertSystem, LegacyAlertAdapter, AccessControlObserver, IncidentLogObserver, Incident, DispatchStrategy (and its concrete classes)

The single call reportIncident orchestrates four subsystem operations (strategy selection, unit dispatch, area lock and alert send). The subsystem classes are independently accessible. A caller can invoke public functions if they choose to do so. The facade owns every subsystem object through a unique_ptr, which is the only thing that wires them together.	
Strategy (new!)	• Strategy (DispatchStrategy) • ConcreteStrategy (HighSeverity, LowSeverity) • Context (CampusGuard) • Client (CampusGuard, through strategyFor)
CampusGuard holds the strategy as a unique_ptr and delegates selectUnits, label and alertMessage to it. The client and context are the same class – the facade chooses and uses the strategy. HighSeverity dispatches all three teams and sends an EVACUATE alert and LowSeverity dispatches Security only and sends a LOCKDOWN alert.	
Observer (new!)	• Subject/ConcreteSubject (Incident) • Observer (IncidentObserver) • ConcreteObserver (AccessControlObserver, IncidentLogObserver)
Subject and ConcreteSubject are merged into one class as a deliberate simplification to prevent over-engineering and unnecessary bloat of the design without losing pattern fidelity, there is only one kind of observable entity in the system. If multiple types needed observation, an abstract Subject base class would be extracted. The subject has a non-owning list of observers, since observers are owned by CampusGuard, not by the incident, so they outlive individual incidents and serve all incidents. The push model has been used – the subject passes *this into onStatusChange, observers do not need a back-reference.	

Justification for Adding Observer and Strategy

Observer was chosen because “an unknown/growing number of things care when an incident’s status changes” which is a broadcast problem, not a coordination problem. We considered certain alternatives and found them unsuitable, namely:

- State: would replace a switch with transitions inside state objects, but the core issue wasn’t the state changes, but the notification of state changes
- Mediator: made for coordinating colleagues, but the observers aren’t colleagues, they are non-participating listeners.

Strategy was chosen because “which units get dispatched depends on severity, and that logic shouldn’t be in the facade.” We considered certain alternatives and found them unsuitable, namely

- Template Method: would work, but locks the algorithm to a base class which is less flexible than being able to swap strategy object
- State: would fit if the dispatch behaviour changed based on internal state, but the severity (which determines strategy) is an input rather than a state the system moves through

Design Decisions: Associations, Inheritance Relationships, Multiplicities & Ownership Decisions

Ownership is centralised in the facade. CampusGuard owns every subsystem object mentioned in its section of the design pattern and participants table via `unique_ptr` members. Destruction is thus unambiguous. When CampusGuard is destroyed, the entire subsystem is destroyed with no manual delete and no risk of leaks. There is also no risk of accidental copying, so there is only one facade owning the subsystem. Every polymorphic base class declares a virtual destructor, deletion through a base pointer is therefore well defined.

All other relationships are non-owning. Classes hold raw pointers to objects they reference but do not own, for instance:

- Response units reference the coordinator (`IncidentCoordinator*`)
- The coordinator references the concrete teams (`SecurityTeam*`, `MedicalTeam*`, `FacilitiesTeam*`)
- Commands reference their receivers (`ResponseUnit*`, `AccessControlSystem*`, `AlertSender*`, `IncidentCoordinator*`)
- Observers reference the access control system (`AccessControlSystem*`)
- Incidents reference their observers (`vector<IncidentObserver*>`)

None of these relationships participate in destructions. The objects they point to are owned by the facade and outlive the referencer. Typing helps make this distinction clearer, `unique_ptr` being used means ownership whilst a raw pointer means reference.

Incident holds a non-owning collection of observers, which are attached dynamically and can serve multiple incidents. They outlive any single incident and are owned by the facade, not by the incident. An incident can have 0 or more observers. In the event that an incident has been created and no observers have been attached there are no observers, whilst in any other event there is any number of observers.

The invoker owns the commands it has executed via a vector of unique pointers; this enables undo since the invoker holds the command objects, so they can be reversed. The OperatorConsole/invoker can own 0 (when no commands have occurred yet) or more commands.

Facade owns one of each of its subsystems, more than that would not be necessary in this design, there is one of each type of subsystem working within CampusGuard to form a cohesive machine. The only exception to this is incident, where CampusGuard has a vector of incidents which have occurred, so there can be 0 or more incidents. The other exception is DispatchStrategy, CampusGuard owns 0 or 1 DispatchStrategy, since we prevent holding a stale strategy by nulling the strategy at the end of a response, when that happens no strategy is being used.

IncidentResponseDesk holds a non-owning reference to each colleague so it can route events to them.

DispatchUnit holds a non-owning reference to the receiver it acts on.

SendAlert holds a non-owning reference to the adapter interface.

SendAlert holds a non-owning reference to the mediator, allowing it to fire an event after a successful alert.

SecureArea holds a non-owning reference to its receiver AccessControlSystem

SecureArea holds a non-owning reference to IncidentCoordinator, allowing the command to fire an event after a successful lock.

ResponseUnit has an association with IncidentCoordinator through its protected coordinator pattern, defining the Mediator pattern, allowing colleagues to talk to the mediator instead of each other.

We see that the direction of association in colleague-to-colleague, observer-to-subject and command-to-receiver are from active caller to the object it uses. There is also a non-owning reference from ResponseUnit to IncidentCoordinator, and from IncidentCoordinator, through IncidentResponseDesk (which has non-owning references to FacilitiesTeam, SecurityTeam and MedicalTeam), to ResponseUnit.

Inheritance / class hierarchies in the system:

- OperatorCommand → DispatchUnit, SecureArea, SendAlert. The Command pattern requires a common interface so the invoker can treat all commands uniformly (execute, undo, name) and hold them in a single history
- ResponseUnit → SecurityTeam, MedicalTeam, FacilitiesTeam. The Mediator pattern requires colleagues to share a common abstract interface (handleEvent, dispatch, recall) so the mediator can treat them uniformly and route events to any of them
- IncidentCoordinator → IncidentResponseDesk. The mediator interface allows colleagues to hold an abstract pointer, decoupling them from the concrete mediator
- IncidentObserver → AccessControlObserver, IncidentLogObserver. The Observer pattern requires a common interface (onStatusChange) so the subject can notify all observers without knowing their concrete types
- AlertSender → LegacyAlertAdapter. The Target interface allows the client (SendAlert) to hold an abstract AlertSender* and remain unaware of the adapter or the legacy system behind it
- DispatchStrategy → HighSeverity, LowSeverity. The Strategy interface allows the context (CampusGuard) to hold an abstract DispatchStrategy* and delegate dispatch decisions without if/else logic

Inheritance exists in these cases to satisfy a design pattern's requirement for polymorphic dispatch rather than code reuse. Each derived class has its own behaviour whilst the base class exists to define the common interface.

Some key design decisions we made are as follows:

1. The facade is the sole owner
All subsystem objects are owned by CampusGuard and referenced non-owningly by the classes that use them. The ownership model is unambiguous, with one owner for every object.
2. Non-owning raw pointer for cross references
Every cross-pattern reference (colleagues → mediator, commands → receivers, observers → ACS, incident → observers) is a raw pointer. This expresses use without ownership.
3. Circular dependency resolved via setCoordinator
The response units need the coordinator, but the coordinator also needs their teams. To break the circular dependency, CampusGuard constructs the teams first with nullptr coordinators, then constructs the coordinator with the team pointers, then calls setCoordinator on each team. This is the only place where two-phase initialisation is used.
4. Events are named string constants

event names live in an Events namespace in Types.h. This keeps the vocabulary centralised (no typos), readable and extensible (adding an event is a one-line change).

5. Push notification model is used in Observer

The subject passes itself to the observer (`onStatusChange(Incident&)`), rather than the observer storing a back-reference and pulling state. This keeps the observer decoupled from the incident and lets the same observer instance serve any incident it is attached to

6. Strategy is used both for unit selection and alert type

`DispatchStrategy` is not limited to `selectUnits` — it also returns the alert type via `alertMessage()`. This makes the strategy genuinely behavioural: high and low severity produce different dispatch sets and different alert codes, not just different labels

7. Commands own their mediator notification

Both `SecureArea` and `SendAlert` hold a `IncidentCoordinator*` and fire an event after successful execution. This keeps the coordination logic in the commands themselves rather than in the facade, and ensures the mediator is notified only on success (exceptions prevent the notification from firing)

Where the Patterns Interact

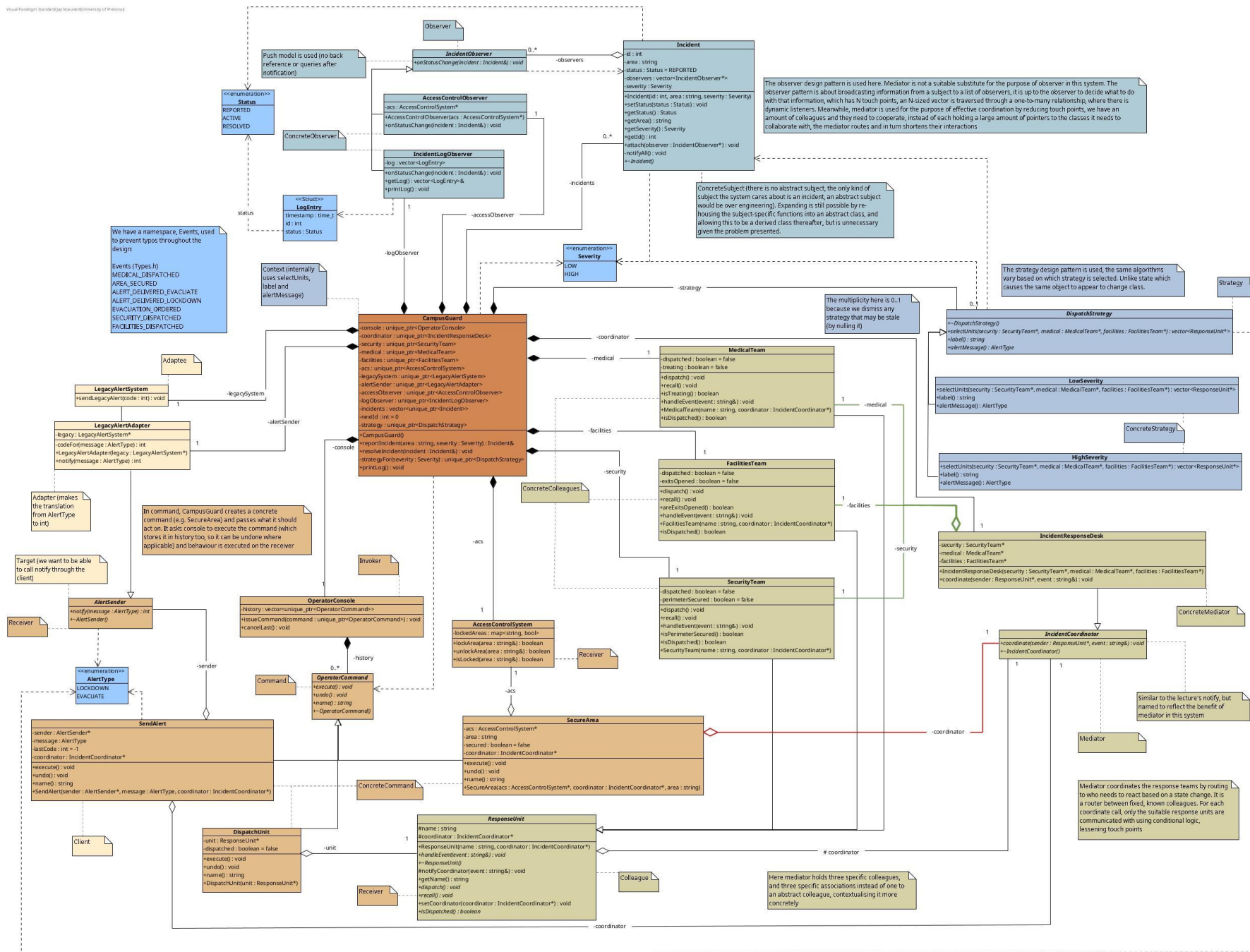
The six patterns interact in a single workflow and `CampusGuard::reportIncident` coordinates all of them.

The Facade provides the entry point. Strategy chooses which units to dispatch and which alert to send. Command represents each dispatch, lock, and alert as an object executed through `OperatorConsole`. The commands themselves trigger Mediator events (e.g. `MedicalTeam::dispatch` → `SecurityTeam::handleEvent`) and reach the Adapter, which translates alert types into legacy codes. Finally, `incident.setStatus(ACTIVE)` fires the Observer chain, which locks the area and records the log.

Each pattern is load bearing: removing Command breaks undo/logging, removing Mediator breaks cross-team coordination, removing Adapter breaks legacy integration, removing Facade breaks the single entry point, removing Observer breaks status propagation, and removing Strategy breaks severity-based dispatch.

Task 4

System UML Class Diagram



Additional UML Diagrams and the Code Used to Model Each

UML Sequence Diagram Showing the Chain Between Command and Mediator

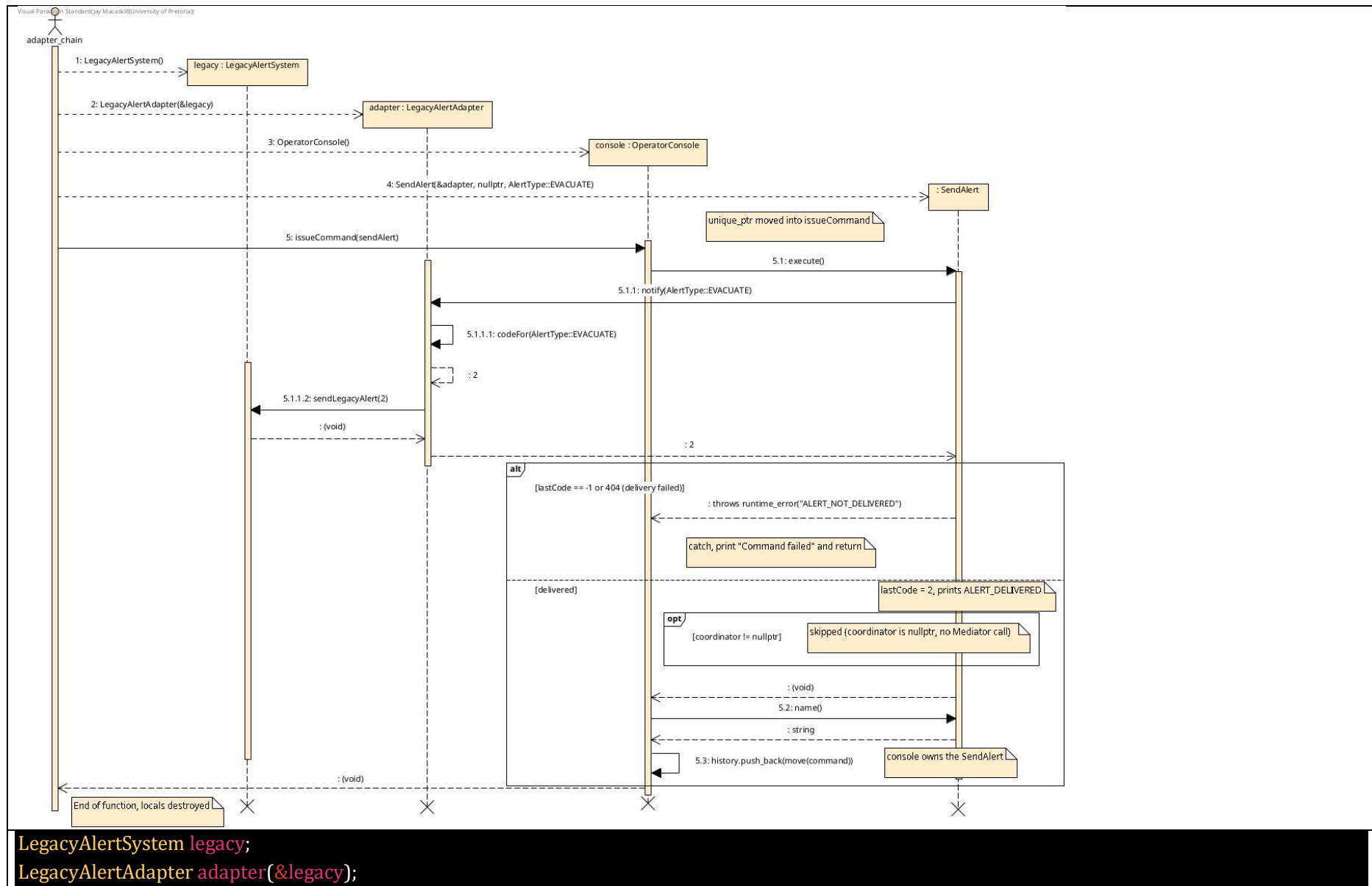
```
SecurityTeam security("Security", nullptr);
MedicalTeam medical("Medical", nullptr);
FacilitiesTeam facilities("Facilities", nullptr);
IncidentResponseDesk coordinator(&security, &medical, &facilities);

security.setCoordinator(&coordinator);
medical.setCoordinator(&coordinator);
facilities.setCoordinator(&coordinator);

OperatorConsole console;

console.issueCommand(unique_ptr<DispatchUnit>(new DispatchUnit(&security)));
console.issueCommand(unique_ptr<DispatchUnit>(new DispatchUnit(&medical)));
```

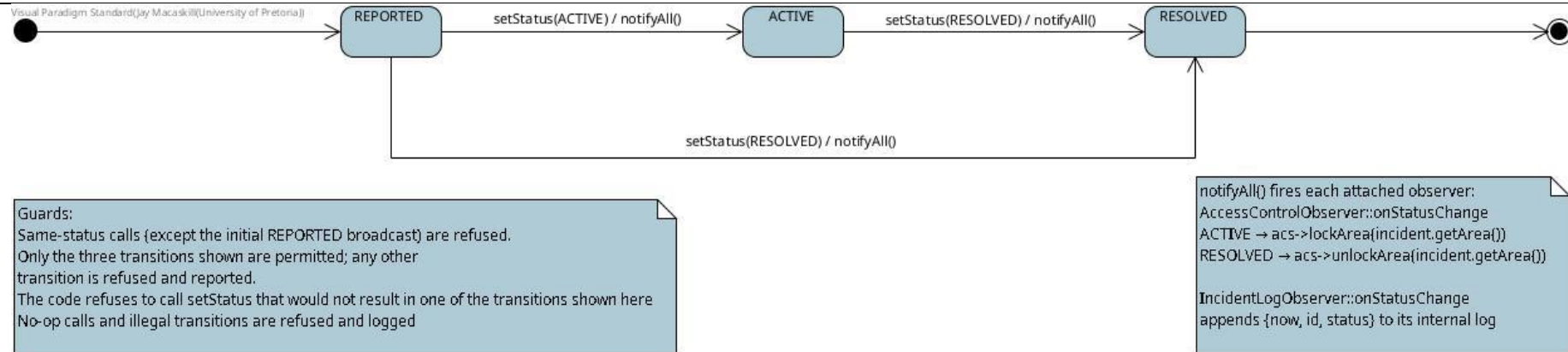
UML Sequence Diagram Showing the Adapter Chain



```
OperatorConsole console;
```

```
console.issueCommand(unique_ptr<SendAlert>(  
new SendAlert(&adapter, nullptr, AlertType::EVACUATE)));
```

UML State Diagram Showing the Lifecycle of an Incident



We chose a state diagram to explain the Observer pattern because Observer exists to react to a change of state, and this diagram shows the two things the pattern depends on: what counts as a change, and what happens when one occurs. The three states (REPORTED, ACTIVE, RESOLVED) are the subject's state (Incident::status), and every valid transition carries the effect / notifyAll(), so the diagram shows exactly when the subject broadcasts. Its guards show when it does not: repeated statuses and illegal transitions are refused and no notification is sent, so observers never receive duplicate or invalid updates. The accompanying note then shows how each concrete observer reacts to the same broadcast (AccessControlObserver locks the area on ACTIVE and unlocks it on RESOLVED, while IncidentLogObserver records a timestamped entry each time), which makes the independence of the observers visible. A sequence diagram would show the attach/notify message flow, but it would treat the status as a bare argument. The state diagram explains why the notification exists and when it is valid to fire, which is the design problem Observer solves in CampusGuard. The observer structure (Incident → 0..* IncidentObserver) is on the class diagram.

```
Incident incident(99, "Demo Area", Severity::HIGH);
```

```
AccessControlSystem acs;
```

```
AccessControlObserver accessObserver(&acs);
```

```
IncidentLogObserver logObserver;
```

```
incident.attach(&accessObserver);  
incident.attach(&logObserver);  
  
incident.setStatus(Status::ACTIVE);  
  
incident.setStatus(Status::RESOLVED);
```

Task 5

[Valgrind Evidence for the Final Interactive Demo](#)

```
• > docker compose run --rm campusguard make valgrind-demo
Container practical_5-campusguard-run-82081c1cb901 Creating
Container practical_5-campusguard-run-82081c1cb901 Created
valgrind --leak-check=full --show-leak-kinds=all ./demo
==7== Memcheck, a memory error detector
==7== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==7== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==7== Command: ./demo
==7==
==7==
==7== HEAP SUMMARY:
==7==    in use at exit: 0 bytes in 0 blocks
==7==   total heap usage: 265 allocs, 265 frees, 91,902 bytes allocated
==7==
==7== All heap blocks were freed -- no leaks are possible
==7==
==7== For lists of detected and suppressed errors, rerun with: -s
==7== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

[Valgrind Evidence for the Static Testing Main](#)

```
• > docker compose run --rm campusguard make valgrind
Container practical_5-campusguard-run-59f63cc3ef67 Creating
Container practical_5-campusguard-run-59f63cc3ef67 Created
valgrind --leak-check=full --show-leak-kinds=all ./campusguard
==7== Memcheck, a memory error detector
==7== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==7== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==7== Command: ./campusguard
==7==
```

```
✓ TESTS COMPLETE ✓
==7==
==7== HEAP SUMMARY:
==7==    in use at exit: 0 bytes in 0 blocks
==7==   total heap usage: 271 allocs, 271 frees, 88,836 bytes allocated
==7==
==7== All heap blocks were freed -- no leaks are possible
==7==
==7== For lists of detected and suppressed errors, rerun with: -s
==7== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

GDB Investigation: Uncaught Exception During Facade Integration Testing

While implementing the different patterns, we tested each one in main.cpp through its own test function. This allowed us to test patterns in isolation, assuming they didn't depend on others. Facade was therefore the last pattern we tested, because it depends on all the others.

We corresponded with each other as each pattern was completed, and at the time we tested the Facade, everyone had reported their patterns as done. However, when we ran main, the program crashed unexpectedly with the message:

```
✨ [STARTING FACADE TESTS] ✨

[TEST HIGH SEVERITY]

🚨🚨🚨 New Incident #0 reported @ Library 🚨🚨🚨
[Incident 0] already in this state, no notification sent
🕒 Dispatch strategy selected: High-severity: All response teams dispatched (Security + Medical + Facilities)
terminate called after throwing an instance of 'char const*'
[1] 114779 abort (core dumped) ./campusguard

~ /De/COS214_prac5/p/practical_5 facade 12 726
```

At first glance we didn't know what was going on. Rather than hunting for the issue by reading code, we used GDB to investigate.

```
> gdb ./campusguard
GNU gdb (GDB) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./campusguard...
(gdb) run
Starting program: /home/finn/Desktop/COS214_prac5/practicals/practical_5/campusguard

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.archlinux.org>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for /lib64/ld-linux-x86-64.so.2
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/libthread_db.so.1".
```

GDB confirmed that the program aborted inside a throw statement.

```

🚨🚨 New Incident #0 reported @ Library 🚨🚨
[Incident 0] already in this state, no notification sent
🚚 Dispatch strategy selected: High-severity: All response teams dispatched (Security + Medical + Facilities)
terminate called after throwing an instance of 'char const*'

Program received signal SIGABRT, Aborted.
0x00007ffff789a29c in ?? () from /usr/lib/libc.so.6
(gdb) █

```

The backtrace showed that the throw originated in `OperatorConsole::issueCommand`, called from `CampusGuard::reportIncident`, called from `test_facade`.

```

Program received signal SIGABRT, Aborted.
0x00007ffff789a29c in ?? () from /usr/lib/libc.so.6
(gdb) backtrace
#0 0x00007ffff789a29c in ?? () from /usr/lib/libc.so.6
#1 0x00007ffff783e7d0 in raise () from /usr/lib/libc.so.6
#2 0x00007ffff7825681 in abort () from /usr/lib/libc.so.6
#3 0x00007ffff7c9ac50 in ?? () from /usr/lib/libstdc++.so.6
#4 0x00007ffff7cb673a in ?? () from /usr/lib/libstdc++.so.6
#5 0x00007ffff7c9a5e9 in std::terminate() () from /usr/lib/libstdc++.so.6
#6 0x00007ffff7cb69f6 in __cxa_throw () from /usr/lib/libstdc++.so.6
#7 0x000055555556a56b in OperatorConsole::issueCommand (this=0x55555558e960,
command=std::unique_ptr<OperatorCommand> = {...}) at OperatorConsole.cpp:21
#8 0x0000555555559b10 in CampusGuard::reportIncident (this=0x7fffffd030, area="Library",
severity=Severity::HIGH) at CampusGuard.cpp:57
#9 0x0000555555567cc7 in test_facade () at main.cpp:146
#10 0x0000555555567ed0 in test () at main.cpp:183
#11 0x0000555555567f09 in main () at main.cpp:193
(gdb) █

```

When we looked at `issueCommand`, we found that it was still the Visual Paradigm-generated stub. It had been missed during implementation. The fix was to implement `issueCommand` with proper command execution and exception handling, and to review the other files for any remaining stubs. After the fix, the Facade test ran to completion without crashing.

```
void OperatorConsole::issueCommand(unique_ptr<OperatorCommand> command) {  
    throw "Not yet implemented";  
}
```

```
void OperatorConsole::issueCommand(unique_ptr<OperatorCommand> command)  
{  
    try  
    {  
        command->execute();  
        cout << "✅ Executed: " << command->name() << endl;  
    }  
    catch (const exception& e)  
    {  
        cout << "❌ Command failed: " << e.what() << endl;  
        return;  
    }  
    history.push_back(move(command));  
}
```

Docker Run Instructions

Requirements

- Installed Docker and running it
- Installed Docker Compose

Build and launch the demonstration

docker compose up --build

This builds the image from the Dockerfile and starts the container. The interactive demo begins automatically and runs statically without user input.

Run the static tests

```
docker compose run --rm campusguard ./campusguard
```

Run the demo interactively

```
docker compose run --rm campusguard ./demo
```

Debug with GDB

```
docker compose run --rm campusguard bash  
    gdb ./campusguard # static  
    gdb ./demo       # interactive
```

Check for memory leaks with Valgrind

```
docker compose run --rm campusguard make valgrind      # static scenarios  
docker compose run --rm campusguard make valgrind-demo # interactive demo
```

```
# Dockerfile
```

```
FROM ubuntu:22.04
```

```
ENV DEBIAN_FRONTEND=noninteractive
```

```
RUN apt-get update && apt-get install -y g++ make gdb valgrind git && apt-get clean && rm -rf /var/lib/apt/lists/*
```

```
WORKDIR /campusguard
```

```
COPY . /campusguard/
```

```
RUN make clean && make
```

```
CMD ["sh", "-c", "echo demo | ./demo"]
```

```
# docker-compose.yml
services:
  campusguard:
    build: .
    image: campusguard:latest
    stdin_open: true
    tty: true
```

Collaboration Reflection

The team divided work by pattern and cycled paired programming roles during integration and review as far as possible. During implementation, each implementer was navigated by the eventual reviewer and the tester role rotated, Mohammad tested Strategy, Adapter and Command, Emmanuel tested Observer and Jay tested Mediator and Facade.

We collaborated across four phases (design and infrastructure, parallel pattern implementation, integration and review and documentation).

One member drove the initial design (UML, class skeletons, Makefile) while the other two navigated by contributing student details and reviewing design decisions as they landed. Establishing the foundation subsequent work built on.

We split the six patterns by member. Mediator and Facade, Strategy and Adapter, Observer and Command. Each pattern was developed on its own feature branch off the shared integration branch. The driver/navigator split was active during this phase: implementers worked independently, then submitted their work for navigation by another member who reviewed the output, output formatting, and correctness before merging into the shared integration branch.

Then, all feature branches were merged into the shared integration branch through a series of focused merges. During this phase, roles cycled: the same member who drove Mediator and Facade also drove the integration, acting as navigator for the

other patterns — reviewing the Command implementation (output styles), Observer implementation (try/catch positions and output), Adapter implementation (output intuitiveness), and Strategy implementation (output). Testing during this phase was distributed: the Strategy, Adapter, and Command tests were written by the second member; the Observer test by the third; and the Mediator and Facade tests by the first. An integration bug during Facade testing was investigated with GDB, leading to the discovery of an unimplemented method and a documented fix.

The PDF sections in this document were then assembled and completed, with members reviewing each other's work throughout.

The distribution of collaboration is reflected by the Git history, where we see the lines written by each member and the specific sections they contributed to.

Team Member Contribution Statement

Jay Macaskill – 25198387

- Initial system design (UML class diagram, GoF participant mapping and related PDF items such as Task 1)
- Header and skeleton files
- Makefile
- Docker setup
- Merge conflict resolutions as needed
- Facade and Mediator patterns (implementation and testing)
- Code reviews
- GDB investigation
- Proposed branching workflow
- README
- State Diagram
- Reviewing of work such as diagrams

Mohammad H Jafari – 25312040

- Strategy code
- Adapter code
- Command code
- Testings in main
- UML Sequence Diagram for Adapter
- I also assisted in creating the submission PDF file

Emmanuel Boateng – 23586975

- Command & Observer patterns during implementation phase
- Observer testing in main
- UML Sequence Diagram for Command & Mediator